一、选择题 1.当存储空间有足够的空闲空间时，在保持表内元素顺序，相对不变的情况下，下列哪些操作会必然导致产生移动次数（这考的是 c 语言数组的内存复制的知识）Ⅰ. 在表头插入一个元素Ⅱ. 在表头删除一个元素Ⅲ. 在表尾插入一个元素Ⅳ. 在表尾删除一个元素 A. Ⅰ、ⅡB. Ⅰ、ⅢC. Ⅱ、ⅣD. Ⅲ、Ⅳ

2. 设有一个双向链表 L,结构为 [p2,d,p1],头结点为 head,初始时 head=cu。现要将每个结点的 p2 指向 p1 指向结点的直接后继，应该进行的操作是 ()

A. while (cu != NULL) {cu->p2 = cu->p1->p1; cu = cu->p1;}

B. while (cu != NULL && cu->p2 != NULL) {cu->p2 = cu->p1->p1; cu = cu->p1;}

C. while (cu != NULL) {if (cu->p1 != NULL) {cu->p2 = cu->p1->p1; cu = cu->p1;}}

D. while (cu != NULL) {if (cu->p1 != NULL) cu->p2 = cu->p1->p1; cu = cu->p1;}

分析：需要判断 cu->p1 是否为空才能访问 cu->p1->p1，但每次都要移动 cu = cu->p1。选项 D 的逻辑正确。

3. 已知二叉树 T 的中序序列为 {b,c,d,f,c,a,g}，层次遍历序列为 {a,b,g,c,d,e,f}，则后序遍历序列为（ ）A. c,e,d,f,b,g,aB. c,e,f,d,b,g,aC. e,f,d,c,b,g,aD.

e, g, f, d, b, c, a 根据中序和层次遍历重建二叉树，然后求后序遍历

4. 森林中有 5 棵树，其结点个数分别为 2、3、4、5、7，森林中树的次序可以任意，问二叉树的最小高度为（）A. 5B. 6C. 8D. 105 棵树，节点数 2,3,4,5,7，总共 21 个节点。 最小高度策略：将节点多的树放前面作为兄弟。 最优排列：7,5,4,3,2，高度为 6。

5. 已知字符串 abcdefg 对应权值为 1,2,4,5,8,10,12，使得带权路径长度最小，与 e 同层的结点有（）A. dB. gC. d 和 fD. f 和 g 权值：a=1, b=2, c=4, d=5, e=8, f=10, g=12 构建哈夫曼树，e(权值 8)与 f(权值 10)在同一层。

6. 采用邻接表存储有向图 G，求图 G 一个顶点入度的时间复杂度为（）A. O (|V|)B. O (min (|V|, |E|))C. O (|E|)D. O (max (|V|, |E|))邻接表存储有向图，求一个顶点入度需要遍历所有邻接表，时间复杂度 O(|V|+|E|) = O(max(|V|,|E|))

7. 设有向图中顶点数为 n，只有一个初始顶点 S，有多个标记顶点 T，图中每条边有一个字符，S 到 T 的所有字符串构成集合 S，则以下说法中错误的是（）A. 若 G 无环，S 为有穷集合B. 若 G 无环，S 有长度等于 n 的串C. 若 G 有环，S 有长度大于 n 的串D. 若 G 有环，S 有小于 2n 的串若 G 无环，从 S 到 T 的路径长度最多为 n-1（不可能等于 n）。

8. 在高度为 4 的平衡二叉树中，根的左、右子树结点数相差最多的是（）A. 1B. 2C. 3D. 5 高度 4 的 AVL 树根节点左

右子树高度差为 1（3 和 2），最大节点数差为高度 3 满二叉树（7 个）与高度 2 最小 AVL 树（2 个）之差，即 5，选 D。

9. 使用直接插入排序对序列进行升序排序，以下比较次数最少的是（）A. 30,27,56,41,80,95,69B. 31,43,26,55,63,99,77C. 61,84,51,23,34,91,40D. 93,32,48,81,50,21,72

10. 有两门科目，成绩分别为 C1、C2，由成绩 C1、C2 和总成绩保存在数组 M 中。排名是按照总成绩由高到低，总成绩相同的按照 C1 的乘积由高到低，C1 相同的按照原始次序排名。对数组 M 先按照 C1，再按照总乘积排序，这种排序算法是（）A. 基数排序 B. 快速排序 C. 希尔排序 D. 选择排序按 C1 和总成绩分别排序，保持原始次序，是基数排序的特点。

11. 使用 k 路归并对外存中的数据进行排序，归并趟数为 d，以下说法正确的是（）Ⅰ. 增大 k 可以减少 d 的值Ⅱ. 归并趟数 d 不受初始归并段影响Ⅲ. 可用内存的大小限制初始归并段的长度 A. Ⅰ B. Ⅰ、Ⅱ C. Ⅰ、Ⅲ D. Ⅱ、Ⅲk 路归并趟数 d 与 k 成反比，初始归并段数影响 d，内存大小限制段长度，Ⅰ、Ⅲ正确，选 C。

12. 关于计算机的系统层次的叙述，错误的是（）A. 最上层是应用软件层 B. 指令集体系统结构是软件和硬件的接口 C. 计算机组成（即微架构）属于指令集体系统结构的物理实现层 D. 操作系统可通过 ISA 进行抽象，向上层软件提供服务计算机组成是指令集体系结构（ISA）的逻辑实现，而非物理实现层，选项 C 表述错误。

13. 对机器数 10100110B 先算术右移 3 位再算术左移 2 位，最终结果是（）A. 1101 0000BB. 1101 0011BC. 0101 0000BD. 0101 0011B10100110B → 算术右移 3 位 → 11110100B → 算术左移 2 位 → 11010000B

14. 已知 float 型变量用 IEEE754 单精度浮点数表示，采用就近舍入（中间值取偶数），若 float 型变量 x 为 12.1，则 x 的机器数是（）A. 41419999HB. 4141999AHC. 41E0 CCCCHD. 41E0 CCCDH12.1 转换为 IEEE754 单精度浮点数，经就近舍入（中间值取偶数）处理后，机器数为 4141999AH，选 B。

15. 用 8 个 64M×8 位的 DRAM 芯片按交叉编上方式构成主存储器，并与一个宽度为 64 位的存储总线相连，主存每次最多每次读写 64 位，且按字编址，则下列地址中，与主存地址 0018 001DH 位于同一芯片中的是（）A. 0000 01D5HB. 000F A020HC. 0018 001EHD. 0F02 0014H

16. 下列不是由指令集体系统结构规定的是（）

A. 输入输出指令

B. 采用向量中断

C. 虚拟存储管理方式

D. 指令流水线是否使用超级流水线技术这是微架构层面的实现细节，不由 ISA 规定。

17. 下列指令中执行后有可能按存放顺序执行其下一条指令的是（）

Ⅰ 条件跳转指令

Ⅱ 过程调用指令

Ⅲ 陷阱（Trap）指令

Ⅳ 过程返回指令

A. 仅 Ⅰ、Ⅱ

B. 仅 Ⅰ、Ⅳ

C. 仅 Ⅱ、Ⅲ

D. 仅 Ⅲ、Ⅳ条件跳转不满足条件时顺序执行，过程返回指令在特殊内存布局下可顺序执行，Ⅰ、Ⅳ 符合，选 B。

18. 某计算机按字节编址，数据 cache 共有 1024 行，采用 4 路组相联映射，主存块大小为 32B，若访问主存地址为 1028 的 4 字节数据，则该数据所在主存块对应的组号为（）

A. 4

B. 16

C. 32

D. 64 组数 = 1024/4=256，块内地址 5 位，组号 =(1028>>5)%256=32

19. 某计算机按字节编址，虚拟地址为 16 位，页大小为 256B，页表项中包含装入位（P），页框号（PPN）等字段，TLB 采用 4 路组相联映射，共有 16 个页表项，TLB 表项中包含标记（Tag）/ 有效位（V）等字段，在 TLB 表项与主存页表同步后，

若主存页表中页号 22 对应的页表项中 P=0，PPN=2AH，则下列不可能出现在组号为 2 的 TLB 表项中的是（）

A. Tag=05H，V=1，PPN=1CH

B. Tag=06H，V=1，PPN=2AH

C. Tag=16H，V=0，PPN=2AH

D. Tag=1AH，V=0，PPN=1CH

20. 在不考虑异常中断处理和访存的额外开销下，下列关于数据通路结构与 CPI 之间的关系是正确的为（）

I 单周期数据通路计算机的 CPI 等于 1

II 多周期数据通路计算机的 CPI 大于 1

III 流水线数据通路计算机的 CPI 等于 1

A. 仅 I、II

B. 仅 I、III

C. 仅 II、III

D. I、II、III

21. 在 I/O 子系统，由驱动程序和中断服务程序直接控制外设与主机之间的输入 / 输出操作，其中需要用到一些特权指令。下列指令中，不是特权指令的是（）

A. I/O 指令

B. 关中断指令

C. 中断返回指令

D. 系统调用指令系统调用是用户态的陷阱指令，不是特权指令。

22. 中断控制 I/O 方式下，实现 I/O 需要硬件和软件协同完成，中断响应和处理过程中所包含的下列工作中，必须由硬件完成的是（）

A. 开中断

B. 中断判优

C. 保存断点

D. 保存通用寄存器中断响应过程中，保存断点的操作由硬件自动完成，开中断、中断判优、保存通用寄存器可由软件实现

23. 下列操作中，在内核态执行的是（）

A. 编译程序

B. 链接程序

C. 装入程序

D. 命令解释程序装入程序涉及内存地址映射和权限管理，需运行在内核态，其余程序均运行在用户态

24. 在支持虚拟存储器系统下的指令执行过程中，正确的是（）

A. 地址转换由操作系统完成

B. 页表项的内容由编译器确定 C.虚拟地址的长度必须与物理地址长度相同D. 异常由操作系统处理虚拟存储器的地址转换由 MMU 硬件完成，异常由操作系统处理

25. 下列的线程描述中，正确的是（）

A. 内核级线程和用户级线程都由操作系统创建

B. 多个内核级线程可以映射到一个用户级线程 C. 同一个进程下的多个内核级线程共享进程栈

C. 同一个进程下的多个线程共享进程堆同一进程内的所有线程共享进程堆资源，线程栈为线程私有

26. 系统中有 8 个进程，执行右图的操作，资源 S 的初始值为 5，若此时 S 的值为 - 2，m 表示执行到访问资源的进程个数，n 表示阻塞的进程个数，则 m 和 n 的值是（）

wait（S）

访问资源

signal（S）

A. 5,2

B. 5,1

C. 6,2

D. 7,1

27. 假设进程 P 的读、写进程集合分别是 R(P) 和 W(P)，进程 Q 的读写进程集合分别为 R(Q) 和 W(Q)，则进程 P 和 Q 并发执行中，不会发生错误的并发执行冲突的充要条件是（）

I R(Q) ∩ W(P) = ∅

II R(P) ∩ R(Q) = ∅

III W(P) ∩ W(Q) = ∅

IV R(P) ∩ W(Q) = ∅

A. I、II

B. I、II、III

C. I、III、IV

D. II、III

28. 若 64 位的系统采用三级虚拟分页存储管理方式，其结构如下图所示，第三级页表所占的页框数是（）



A. 512

B. 32K

C. 256K

D. 128M

29. 下列方法中能够有效降低系统平均访问时间的是（）

I TLB

II 多级页表

III 工作集概念

IV 页表缓冲队列

A. Ⅰ、III

B. II、III

C. Ⅰ、III、Ⅳ

D. Ⅰ、II、ⅣTLB 加速地址转换，工作集减少缺页率，页表缓冲队列缩短访问时间，Ⅰ、III、Ⅳ 均可降低平均访问时间，选 C。

30. 进程 P1 和 P2 共享一个文件 R，该文件的页表项分别是 R1 和 R2，其在 2 个进程中的虚拟地址分别是 W1 和 W2，则下列说法中正确的是（）

A. 页表项 R1 和 R2 的内容完全不同

B. W1 和 W2 映射的物理地址相同

C. 进程 P1 对 W1 的修改不会影响 P2 对 W2 的访问

D. W1 和 W2 虚拟地址相同共享文件的物理存储唯一，因此不同进程的虚拟地址 W1、W2 映射的物理地址相同

31. 下列关于驱动程序的描述中，错误的是（）

A. 驱动程序是硬件与操作系统之间的接口程序

B. 驱动程序需根据硬件特性定制开发

C. 驱动程序需要设置统一的接口

D. 字符设备，块设备都是同一种 I/O 方式字符设备（如鼠标、键盘）采用**流式 I/O**，按字节流传输数据；块设备（如磁盘、U 盘）采用**块级 I/O**，按固定大小的块传输数据，二者 I/O 方式差异显著。

32. 下列操作中，鼠标中断处理程序完成的是（）

A. 解析鼠标的输入指令含义

B. 将鼠标数据同步到用户应用程序缓冲区

C. 将数据从输入设备传输到数据寄存器

D. 将数据从数据寄存器传输到内核缓冲区选项 D（正确）：鼠标触发中断后，中断处理程序的核心工作是读取数据寄存器中的鼠标数据，并将其写入内核缓冲区暂存，供后续处理。

33. 下列关于分层网络体系结构的叙述中，错误的是（）

A. 每层都有明确的功能边界

B. 层次越多效率越高

C. 有利于各层技术独立演化

D. 上层无需关心下层的具体实现细节层次并非越多越好，每增加一层就会引入一次**封装 / 解封装的开销**，过多层次会显著降低数据传输效率。

34. 若在带宽 200kHz，信噪比 S/N=1023 的信道上，发送一个长度为 1500B 的分组，则发送该分组的传输时延至少是（）

A. 1ms

B. 2ms

C. 3ms

D. 6ms

35. 假设采用 CSMA/CA 的 IEEE802.11 无线局域网，其数据传输速率为 300 Mbps，DIFS=128μs，SIFS=28μs。忽略除数据帧以外的其他帧的传输时延及信号传播时延，主机 H 发送一个总长度为 1500B 的数据帧，则 H 从开始发送数据帧至确认接收收到所需的时间至少（）

A. 40μs

B. 68μs

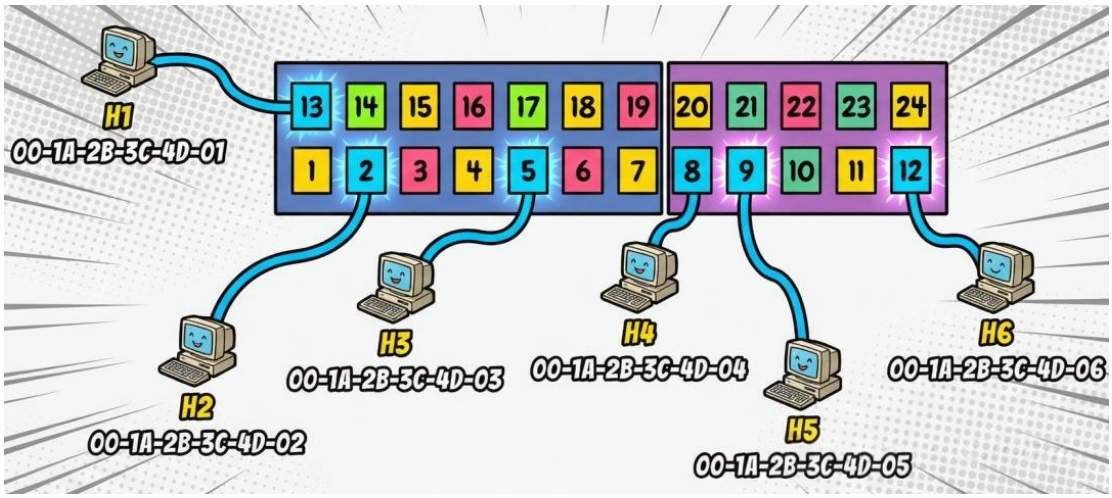
C. 168μs

D. 200μs

36.支持 VLAN 划分的以太网交换机，已按端口划分了两个 VLAN。VLAN 划分结果及部分端口连接主机的 MAC 地址如图所示。下列具有不同目的 MAC 地址（DA）和源 MAC 地址（SA）的以太网帧 F1-F4 中，H3 会接收到的是（

F1：（DA）00-1A-2B-3C-4D-03；（SA）00-1A-2B-3C-4D-01F2：（DA）00-1A-2B-3C-4D-04；（SA）00-1A-2B-3C-4D-05F3：（DA）FF-FF-FF-FF-FF-FF；（SA）00-1A-2B-3C-4D-02F4：（DA）00-1A-2B-3C-4D-06；（SA）

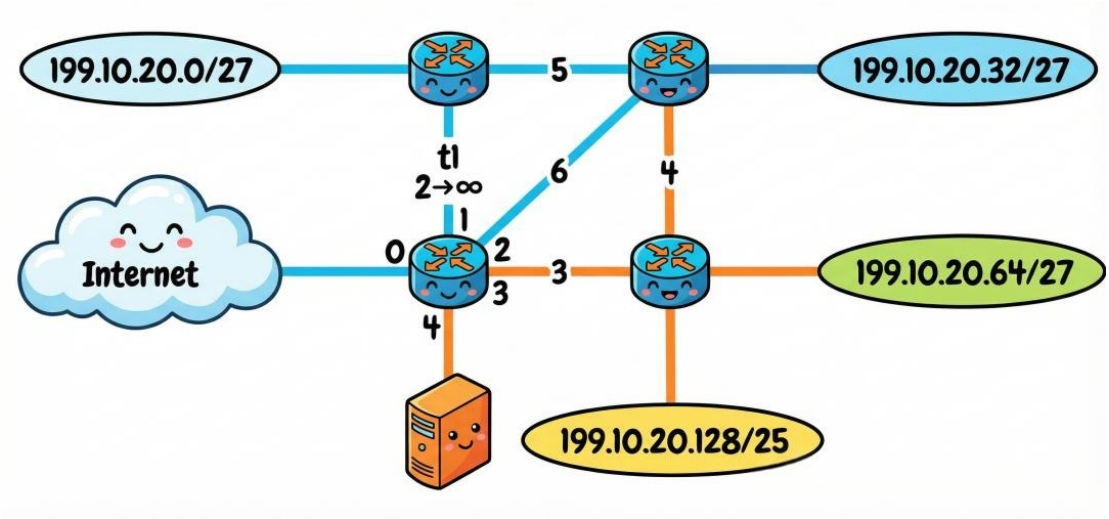
00-1A-2B-3C-4D-03



A. 仅 F2、F

B. 仅 F1、F3C.仅 F1、F2D.仅 F3、F

37.某网络在 t0 刻的网络拓扑和 R1 的路由表如下图所示。R1~R4 是路由器，基于链路状态路由算法计算路由。S0~S4 是路由器 R1 的接口，链路上的数值为链路费用，若 t1（t1>t0）时刻，R1 检测到 R1 到 R2 的链路断开，则 R1 再重新计算路由并进行充分路由聚合后的表中路由项的数量（



A.

B.

D. 6 选

个

38. 下列路由协议中，能将一个自治系统划分为多个区域内的内部网关协议是

）

Ⅰ. OS

F

Ⅱ. R

P

Ⅲ. B

P

A. 仅

Ⅰ

B. 仅

Ⅱ

C. 仅 Ⅰ

Ⅲ

D. 仅 Ⅱ、ⅢOSPF 是支持自治系统内区域划分的内部网关协议，RIP 不支持，BGP 为外部网关

议

39.若将 IP 网络 123.4.4.0/22 划分为规模均衡的 32 个子网，则 IP 地址 123.4.5.11 所在的子网是

）

A. 123.4.4.0/

7

B. 123.4.4.32/

7

C. 123.4.5.0/

7

D. 123.4.5.192/

7

40.下列叙述中不属于 cookie 的技术典型用途的是

）

A. 用户

踪

B. 个性化

荐

C. 构建虚拟购

车

D. 缩短 web 对象的响应时间Cookie 的典型用途包括用户跟踪、个性化推荐、构建虚拟购物车，无法缩短 Web 对象响应时间，选 D 。

二、综合应用题

（第 41~47 小题，共 70 分）

41. 假定二叉搜索树使用二叉链表存储，存储结构如下：

```
typedef struct BSTNode{
    int data;
    struct BSTNode *left,*right;
} BSTNode;
typedef BSTNode BTNode;
```

给一棵二叉搜索树 T 和整数 K，查找树中关键字与 K 之差的绝对值最小的所有结点，并输出该绝对值与结点中的关键字。

（1）给出算法的基本思想。（4 分）

（2）使用 C/C++ 描述算法思想。（8 分）

核心策略：

- **利用二叉搜索树有序性进行智能搜索，动态维护全局最小差值与对应节点集合**

详细步骤：

① 初始化阶段：

- 设置全局变量 **minDiff = INT_MAX**（最小绝对值差）
- 创建动态结果集 **resultList**（存储所有最优节点）

② 递归搜索策略：

对当前节点 T 执行：

a) 计算当前差值 diff = |T->data - K|

b) 更新结果集（3 种情况）：
- 若 diff < minDiff: 清空 resultList，加入 T，更新 minDiff
- 若 diff == minDiff: 追加 T 到 resultList
- 若 diff > minDiff: 不操作

c) 剪枝搜索（BST 性质优化）：
- 若 K < T->data: 优先搜索左子树，再搜索右子树
- 若 K > T->data: 优先搜索右子树，再搜索左子树
- 若 K == T->data: diff=0 为最优，仍需遍历两侧（可能存在重复值）

算法复杂度分析

复杂度类型	最好情况	最坏情况	平均情况	说明
时间复杂度	O(log n)	**O(n)**	O(log n)	最坏情况需遍历全树（如链状树）
空间复杂度	O(log n)	O(n)	O(log n)	递归栈深度+结果链表

优化效果：

相比盲目中序遍历 O(n)，本算法通过剪枝在平衡树上可达 O(log n)

统　　计　　计　　算　　类

题目 4：二叉搜索树最小差值（统计计算类）

问题描述：给定一棵二叉搜索树，找出树中任意两个不同节点值之间的最小差值。

问题形式化描述：

- 输入：二叉搜索树根节点 root
- 输出：最小节点值差值
- 约束：树至少包含 2 个节点

暴力解法思路： 中序遍历得到有序序列，计算相邻元素的最小差值。

```
void inorderTraversal(TreeNode* root, int* values, int* index) {
    if (root == NULL) return;

    inorderTraversal(root->left, values, index);
    values[(*index)++] = root->val;
    inorderTraversal(root->right, values, index);
}

int getMinimumDifference(TreeNode* root) {
    int values[10000];
    int index = 0;

    // 中序遍历收集所有值
    inorderTraversal(root, values, &index);

    // 计算相邻元素最小差值
    int minDiff = INT_MAX;
    for (int i = 1; i < index; i++) {
        int diff = values[i] - values[i-1];
        if (diff < minDiff) {
            minDiff = diff;
        }
    }

    return minDiff;
}
```

42. (10 分)栈的基本操作有出栈和入栈，将序列 1，2，3，…，n 依次入栈，回答下列问题:

（1）当 n=9 时，可以得到出栈序列{2,3,1,6,4,7,5,9,8}吗？可以得到出栈序列{2,3,1,5,6,7,4,9,8}吗？（2 分）

（2）假设 1，2，3，..，n 组成任意序列的出栈序列 P1，P2，.，Pn，在序列中有 Pi、Pj、Pk（i<j<k），若该出栈序列不能由栈得到，则 Pi、Pj、Pk 的大小关系是？（2 分）

（3）若 n=4，则 2 开头的序列的个数有多少个？（2 分）

（4）若 n = k-1 时，出栈序列总共有 M 个，如果 n =k，那么以 1 开头的出栈序列个数有多少个？以 2 开头的出栈序列有多少个？总共的出栈序列有多少个？（4 分）

（1）{2，3，1，6，4，7，5，9，8}

不能得到（因为要输出 4 时，5 还在栈中压着 4）

（2）不合法出栈序列的特征

皮皮灰答案：Pi > Pk > Pj

合法情况的约束：

如果 Pi < Pj < Pk 正常的递增出栈，合法 ✓

如果 Pk < Pi < Pj Pk 先入栈，Pi、Pj 后入栈，合法 ✓

如果 Pi > Pk > Pj 不合法 X

Pi 先出栈，说明 Pj、Pk 此时还未入栈或已入栈

Pk > Pj 说明 Pk 在 Pj 之后入栈

但 Pk 要在 Pj 之前出栈（k>j），这违反了栈的后进先出原则

（3）答案：5 个

1. 2, 1, 3, 4 ✓
2. 2, 1, 4, 3 ✓
3. 2, 3, 1, 4 ✓
4. 2, 3, 4, 1 ✓
5. 2, 4, 3, 1 ✓

实际上这是 $C_3 = 5$（3 个元素的卡特兰数）

（4）已知条件：n = k-1 时，出栈序列总数为 M

根据卡特兰数性质：$M = C_{k-1}$

当 n = k 时：

① 以 1 开头的序列个数：

以 1 开头 → 1 先出栈 → 剩余 2,3,...,k 共 k-1 个元素

答案：M 个（即 C_{k-1}）

② 以 2 开头的序列个数：

以 2 开头 → 1 入栈，2 入栈，2 出栈 → 1 在栈底，剩余 3,4,...,k

剩余元素数量：k-1 个（1 在栈中 + 3 到 k 共 k-2 个待入栈）

答案：M 个（即 C_{k-1}）

③ 总共的出栈序列个数：卡特兰数递推

$$Ck = \frac{2\ (2k+1)}{k+1} C_{k-1} = \frac{2\ (2k+1)}{k+1} M$$

43.(10 分) 某 16 位计算机 c 按字节编址，通用寄存器 R0~R15 的编号为 0~15，存储器地址为 16 位，采用定长指令字，指令有 R、I 和 M 型，格式及功能说明如下：

格式	4 位	4 位	4 位	4 位	功能说明
R 型	0000	rt	rs/num	op1	R[rt] ← R[rt]op1R[rs]; R[rt] ← R[rt]op1num
I 型	op2	rt	imm8		R[rt] ←R[rt]op2 imm8
M 型	op3	offset	offset		R[0] ← M[R[15] +offset]; M[R[15] +offset] ← R[0]

补充说明：

其中 OP1 为 0001、0010 分别表示加、左移指令；OP2 为 0100 表示加立即数指令；OP3 为 1110、1111 分别表示取数、存数指令。

R[r]表示寄存器 r 中的内容，num 表示移位位数，M[addr]表示存储器地址 addr 中的内容。

请回答下列问题：

(1) 主存单元和通用寄存器的宽度各为多少位？(2 分)

(2) OP1 和 OP2 的编码是否可以相同？OP2 和 OP3 的编码是否可以相同？(2 分)

(3)若 R(2)=ABCDH，R(9)=F001H，则指令 **0000 0010 1001 0001** 执行后，R2 和 R9 中的内容分别是多少？(2 分)

(4)若变量 x、y 均为 16 位带符号整数，在存储器中依次从低地址向高地址连续存放，x 的地址在 R15 中，实现 y=16×2-5 的 4 条指令 I1~I4 如题 43 表所示，写出①~④处的内容 (4 分)

21: ① 0000 0000 0000
22:0000 ② 0010
23: 0100 0000 ③
24: 1111 ④

常 规 题 型

（1）8 位，16 位（2 分）

计算机 C 按字节编址，所以**主存单元的宽度**为 8 位。

通用寄存器宽度：与机器字长一致，为 16 位

（2）op1 和 op2 可以相同，处于不同位置，不冲突；op2 和 op3 不可以相同，处于相同位置，有冲突。

OP1 与 OP2：OP1 位于指令[3:0]位（R 型），OP2 位于[15:12]位（I 型），字段不同，编码可重复

OP2 与 OP3：均位于[15:12]位，若编码相同会导致译码器无法区分 I 型和 M 型指令，产生冲突

（3）R2=9BCEH，R9=F001H

指令解码： 0000 0010 1001 0001

[15:12]=0000 → R 型指令 [11:8]=0010 → rt=2 [7:4]=1001 → rs=9 [3:0]=0001 → op1=加法	**执行过程：** • 功能：R[2] ← R[2] + R[9] • 计算：ABCDH + F001H = 1 9BCEH • 结果：保留低 16 位 → **9BCEH**（进位丢弃） • R(9)作为源寄存器内容不变

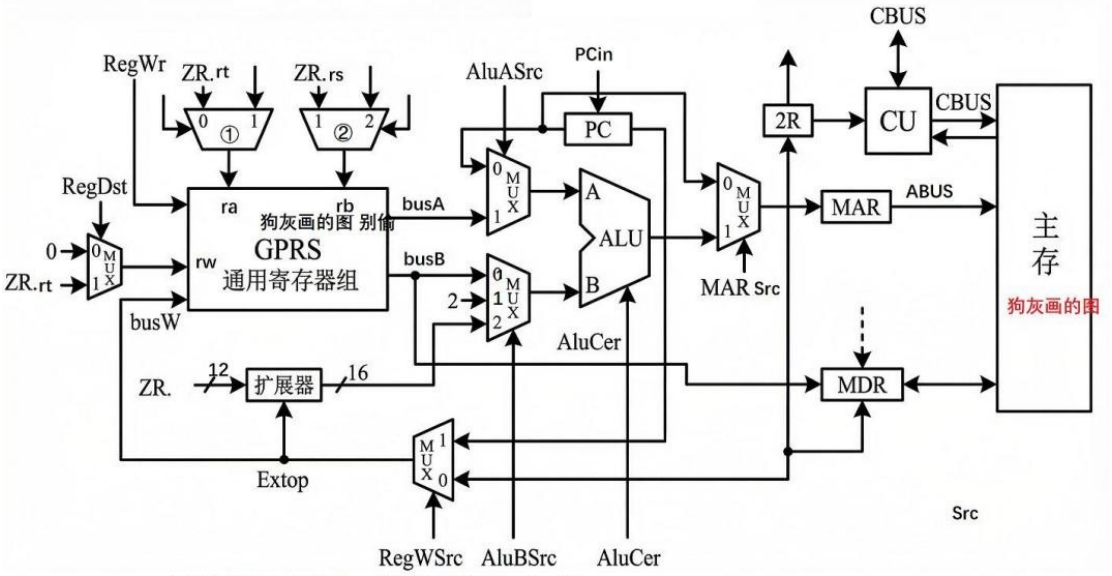
（4）程序填空（实现 y=16×x-5）

指令	填空内容	完整指令	功能说明
I1	① **1110**	1110 0000 0000 0000	M 型取数：R[0]←M[R[15]+0]
I2	② **0000 0100**	0000 0000 0000 0100 0010	R 型左移：R[0]←R[0]<<4
I3	③ **1111 1011**	0100 0000 1111 1011	I 型加法：R[0]←R[0]+(-5)
I4	④ **0000 0000 0010**	1111 0000 0000 0010	M 型存数：M[R[15]+2]←R[0]

技术细节：

- **I1**：offset=0，取 x 到 R0
- **I2**：左移 4 位实现×16（num=0100B=4）
- **I3**：-5 的 8 位补码=1111 1011B=FBH
- **I4**：offset=2（y 地址=x 地址+2，按字编址）

44. （15 分）假定 43 题中计算机 C 的部分数据通路如题 44 所示。

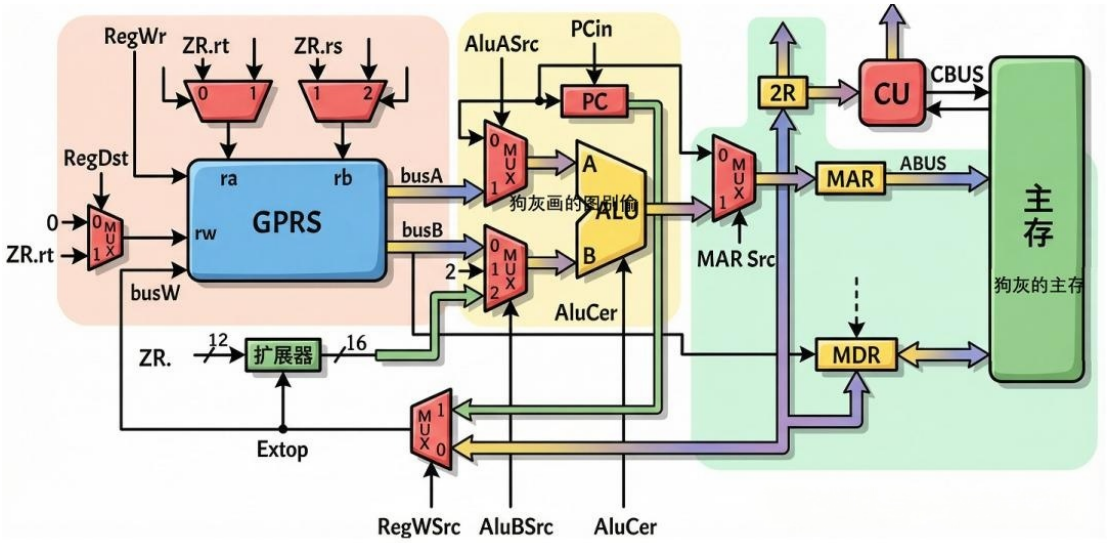


(1) ①②是同一类部件，其名称是什么？（1 分）

(2) I 型指令中 imm8 可以是带符号或无符号整数，M 型指令中 offset 是带符号整数，则 EXTOP 至少有几位？为什么？（2 分）

(3) 取值周期中 MAR SrC、ALUASrC、ALUBSrC、RegWr 的取值各是什么？（4 分）

(4) 左移指令周期中 ALUBSrC、Regwsrc、RegDst、RegWr 的取值各是什么？（4 分）Extop 是否可以与 M 型指令中的 EXtop 相同？为什么？（4 分）



(1)①②是多路选择器

(2) EXTOP 的最少位数

EXTOP 至少需要 1 位。

原因：需要区分两种扩展方式：

0 扩展（用于 I 型指令中无符号的 imm8、左移指令的 num）；

符号扩展（用于 I 型指令中带符号的 imm8、M 型指令的 offset）。

1 位可以表示 2 种状态（如“0”对应 0 扩展，“1”对应符号扩展），满足需求。

(3) 取值周期中 MAR SrC、ALUASrC、ALUBSrC、RegWr 的取值各是什么？（4 分）

取指周期的操作流程：

1. 将 PC 中的指令地址送入 MAR，用于访问主存；
2. 主存根据 MAR 地址，将指令读出并送入 IR；
3. ALU 执行 PC + 1 运算，更新 PC 为下一条指令的地址；
4. 此周期无需写通用寄存器。

MARsrc=0，ALUASrc=0，ALUBSrc=1，RegWr=0

(4) 左移指令周期中各信号的取值及 Extop 的情况

左移指令是 R 型指令

（OP1=0010，操作：R[rt]←R[rt]左移 num，num 来自 IR 的 rs/num 字段）：

ALUBSrc=2： ALU 的 2 端输入选择“扩展器输出的 num”（移位位数）；

RegWsrc=1： 通用寄存器的写数据来源选择“ALU 的运算结果”（左移后的结果）；

RegDst=1： 通用寄存器的写地址选择“IR.rt 字段”（R 型指令的目标寄存器是 rt）；

RegWr=1： 需将左移结果写回寄存器，开启寄存器写使能。

ExtOp 不可以相同，左移指令的 num 是无符号数，需采用零扩展，对应 *Extop*=0；

M 型指令的 offset 是带符号数，需采用符号扩展，对应 *Extop*=1；

二者扩展方式不同，因此 Extop 取值不能相同。

45. 系统采用优先权（优先权越大优先级越大）和时间片轮转调度算法，只有在发生时间中断的时候才运行抢占 CPU，时间中断间隔是 10ms；当进程进入就绪队列时候，时间片是 50ms；当进程是时间片用完重新回到就绪队列时，优先权值的大小减 1；当进程是被更高优先级抢占回到就绪队列时，优先权值的大小不变。优先权相等的时候，先进入就绪队列的被优先调度。4 个进程的到达时刻、初始优先权、CPU 运行时间如下表所示：

进程	到达时刻	优先权	CPU 运行时间
P1	10ms	3	95ms
P2	10ms	4	20ms
P3	12ms	2	40ms
P4	14ms	5	60ms

（1）在 10ms 的时候开始进行进程调度，4 个进程调度结束，中断次数和 CPU 调度次数分别是？P1、P2、P3、P4 在什么时刻进行第一次调度？（5 分）

（2）时间片由 50ms 改为 100ms，CPU 的调度次数会增大、不变或减少？如果中断间隔从 10ms 变为 1ms，则系统开销会增大、不变还是减少？（2 分）

(1) 调度过程模拟

时刻(ms)	事件	就绪队列(优先权)	运行进程	已运行时间	剩余时间	优先权变化	调度次数
10	中断 1：P1、P2 到达首次调度	P1(3)	P2	0	P2:20	-	1
12	P3 到达	P1(3), P3(2)	P2	2	P2:18	-	1
14	P4 到达	P1(3), P3(2), P4(5)	P2	4	P2:16	-	1
20	中断 2：P4 抢占	P2(4), P1(3), P3(2)	P4	0	P4:60, P2:10	P2 不变	2
30	中断 3：P4 继续	P2(4), P1(3), P3(2)	P4	10	P4:50	-	2
40	中断 4：P4 继续	P2(4), P1(3), P3(2)	P4	20	P4:40	-	2
50	中断 5：P4 继续	P2(4), P1(3), P3(2)	P4	30	P4:30	-	2
60	中断 6：P4 继续	P2(4), P1(3), P3(2)	P4	40	P4:20	-	2
70	中断 7：P4 时间片用完	P2(4), P4(4), P1(3), P3(2)	P2	0	P4:10, P2:10	P4:5→4	3
80	中断 8：P2 完成	P4(4), P1(3), P3(2)	P4	0	P4:10	-	4
90	中断 9：P4 完成	P1(3), P3(2)	P1	0	P1:95	-	5
100	中断 10：P1 继续	P3(2)	P1	10	P1:85	-	5
110	中断 11：P1 继续	P3(2)	P1	20	P1:75	-	5
120	中断 12：P1 继续	P3(2)	P1	30	P1:65	-	5
130	中断 13：P1 继续	P3(2)	P1	40	P1:55	-	5
140	中断 14:P1 时间片用完	P3(2), P1(2)	P3	0	P1:45, P3:40	P1:3→2	6
150	中断 15：P3 继续	P1(2)	P3	10	P3:30	-	6
160	中断 16：P3 继续	P1(2)	P3	20	P3:20	-	6
170	中断 17：P3 继续	P1(2)	P3	30	P3:10	-	6
180	中断 18：P3 继续	P1(2)	P3	40	P3:0	-	6
180	P3 完成	P1(2)	P1	0	P1:45	-	7
190	中断 19：P1 继续	-	P1	10	P1:35	-	7
200	中断 20：P1 继续	-	P1	20	P1:25	-	7
210	中断 21：P1 继续	-	P1	30	P1:15	-	7
220	中断 22：P1 继续	-	P1	40	P1:5	-	7
225	P1 完成，系统空闲	-	-	45	P1:0	-	7

中断时刻序列：总中断次数 = 22 次　**CPU 调度次数**　答案：7 次调度

(2) 参数变化影响分析

时间片变化影响

答案：CPU 调度次数会减少

解析：

- 时间片 50ms→100ms，进程单次占用 CPU 时间增加
- 减少因时间片耗尽导致的上下文切换次数
- 示例：P1 原需 2 次调度(50ms×2=100ms)，改为 100ms 后仅需 1 次
- 结论：调度次数↓，CPU 利用率↑

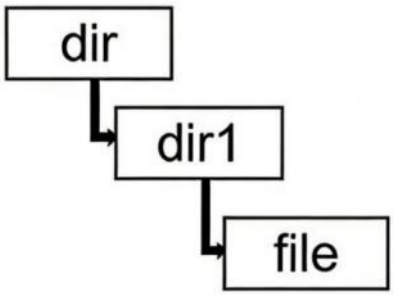
中断间隔变化影响

答案：系统开销会增大

解析：

- 中断间隔 10ms→1ms，中断频率提高 10 倍
- 每次中断需执行：现场保护→调度检查→现场恢复
- 中断次数从 21 次增至约 205 次(215÷1)
- 结论：系统开销↑（中断处理占用更多 CPU 时间），响应性↑

46.(8 分)文件系统的目录项包括文件名和索引节点号，磁盘包含索引节点表、位图、目录、文件数据等元数据。若盘块大小为 4KB，盘号占 4B，索引节点表存放了系统的所有文件（从 0 开始编号），存放在盘块号 100 开始连续的 4096 个盘块中；索引节点占 128B，包含直接地址项 5 个、一级间接地址项 / 二级间接地址项 / 三级间接地址项各 1 个。磁盘位示图和索引节点位示图分别记录磁盘、索引节点的使用情况（0 表示未使用，1 表示使用）。其中目录结构图和文件的索引节点表如下所示，file 文件占 30KB。



文件	索引节点号
dir	100
dir1	201
File	1000

（1）file 的索引节点所在的盘块号是多少？若 file 的索引节点已经读取到内存，要访问 file 文件中偏移地址 21460 的一个字节数据，则最多需要读多少个盘块？如果文件系统中有足够的磁盘空间，则最多可以存放多少个文件？（3 分）

（2）如果要删除目录 dir1，则需要对元数据进行哪些操作？（5 分）

(1) 三问计算

① file 索引节点所在盘块号

计算步骤：

- 每盘块存储索引节点数 = 4KB ÷ 128B = 32 个
- file 索引节点号 1000 在索引节点表中的位置 = 1000 ÷ 32 = 31 余 8
  - 商 31：表示在第 31 个盘块之后
  - 余 8：表示在该盘块内第 8 个位置
- 索引节点表起始盘块号为 100
- file 索引节点所在盘块号 = 100 + 31 = 131

② 访问偏移地址 21460B 需读取的盘块数

寻址分析：

- 直接地址覆盖范围 = 5×4KB = 20480B (0~20479B)
- 目标偏移 21460B > 20480B → 位于一级间接地址块范围
- 一级间接地址内偏移量 = 21460 - 20480 = 980B
- 该偏移量所在数据块编号 = 980 ÷ 4096 = 0 号块
- （一级间接地址项的第 1 个数据块）

访问路径：

```
索引节点(已在内存)
  → 一级间接地址索引块(盘块读取 1)
    → 目标数据块(盘块读取 2)
```

答案：最多需要读取 2 个盘块

③ 系统最多可存放的文件数

计算逻辑：

- 索引节点表占用盘块数 = **4096 个**
- 总索引节点容量 = 4096 盘块 × 32 节点/盘块 = **(2^{17})个**
- 每个文件对应 1 个索引节点

答案：最多可存放 131072 (2^{17}) 个文件

(2) 删除目录 dir1 的元数据操作

阶段一：删除文件 file

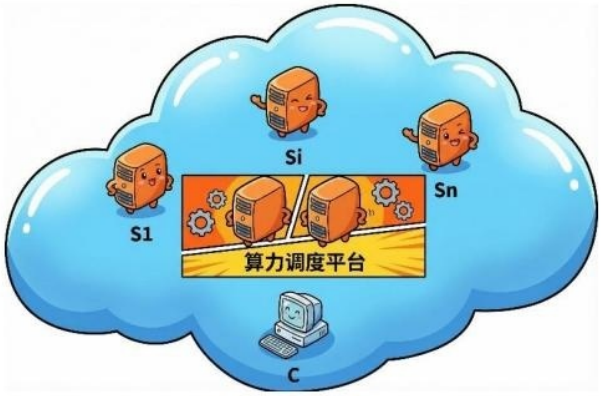
操作步骤	元数据类型	具体操作	数据变化
1. 释放数据盘块	磁盘位示图	将 file 占用的 8 个盘块位设为 0	file 数据区标记为空闲
2. 释放索引节点	索引节点位示图	将索引节点 1000 对应位设为 0	节点 1000 标记为可用
3. 更新父目录	dir1 目录数据块	删除文件名"file"及其节点号 1000 的目录项	dir1 目录项减少 1 条

关键计算： file 占 30KB，需盘块数 = ⌈30KB÷4KB⌉ = 8 个

阶段二：删除目录 dir1

操作步骤	元数据类型	具体操作	数据变化
4. 释放目录盘块	磁盘位示图	将 dir1 目录数据所占盘块位设为 0	dir1 目录区标记为空闲
5. 释放目录节点	索引节点位示图	将索引节点 201 对应位设为 0	节点 201 标记为可用
6. 更新父目录	dir 目录数据块	删除目录名"dir1"及其节点号 201 的目录项	dir 目录项减少 1 条

47.（9 分） 假设 C 建立一条 TCP 连接，向服务器 Si 上传一个总长度为 2000B 的计算任务描述文件。



已知 C 的拥塞窗口初始阈值为 8MSS，MSS=500B，Si 对收到每个 TCP 段进行确认，且确认段不封装数据。接收窗口 rwnd 始终为 1000B，RTT=5ms，C 建立连接时选择的初始序号为 1000，Si 选择的初始序号为 2000，SYN、ACK、FIN 为标志位，seq 为序号，ack_seq 为确认序号。在整个文件传输过程中未出现任何重传或报文丢失。

（1）C 与 Si 建立 TCP 连接过程需要几次握手？C 收到的 SYN=1，ACK=1 的 TCP 段的确认序号是多少？

（2）当 C 接收 Si 发送的 ACK=1，seq=2001，ack_seq=2001，rwnd=1000 确认段后，C 的拥塞窗口增加到多少？C 的发送窗口设置为多少？

（3）C 与 Si 释放 TCP 连接过程需要几次挥手？C 收到最后一个 TCP 报文段的序号（seq），确认序号（ack_seq），FIN 的值分别是多少？

（4）忽略报文段传输时延，且时间从 C 请求建立 TCP 连接时刻算起，则 C 确定 Si 已成功接收到文件的时间是多少？

(1) 三次握手过程与确认序号

握手次数：3 次

三次握手详细流程：

握手阶段	发送方	TCP 段标志	seq	ack_seq	说明
第 1 次	C→Si	SYN=1	1000	-	C 发起连接请求
第 2 次	Si→C	SYN=1, ACK=1	2000	**1001**	Si 确认 C 的 SYN 并发送自己的 SYN
第 3 次	C→Si	ACK=1	1001	2001	C 确认 Si 的 SYN，连接建立

• C 收到 SYN=1, ACK=1 段的确认序号：1001

解析： Si 确认 C 的初始序号 1000，由于 SYN 标志消耗 1 个序号，ack_seq=1000+1=1001

(2) 拥塞窗口与发送窗口计算

C 收到确认段 **ACK=1, seq=2001, ack_seq=2001, rwnd=1000**，说明：

- 该 ACK 确认了 C 发送的第 3 次握手段（seq=1001，无数据）
- 此时 C 开始进入数据传输阶段

拥塞窗口增长过程：

传输阶段	已收 ACK 数	cwnd 变化	计算依据
连接建立后	0 个数据 ACK	1MSS=500B	慢启动初始值
第 1 轮传输后	1 个 ACK	2MSS=1000B	每收到 1 个 ACK，cwnd+1MSS
第 2 轮传输后	2 个 ACK	**3MSS=1500B**	再收 1 个 ACK 继续增加

注意： 题目描述"接收到 seq=2001, ack_seq=2001 确认段后"，该 ACK 确认的是握手阶段报文，此后 C 已发送 2 个数据段并收到确认，故 cwnd 增至 3MSS。

• 拥塞窗口：3MSS = 1500B

• 发送窗口：min{cwnd, rwnd} = min{1500B, 1000B} = 1000B

解析： 发送窗口受限于接收窗口 rwnd=1000B，实际只能发送 1000B（2 个 MSS 段）

(3) 四次挥手过程与最后报文段参数

挥手次数：4 次

挥手阶段	发送方	TCP 段标志	seq	ack_seq	说明
第 1 次	C→Si	FIN=1, ACK=1	3001	2001	C 发送完 2000B 数据后主动关闭
第 2 次	Si→C	ACK=1	2001	3002	Si 确认 C 的 FIN
第 3 次	Si→C	FIN=1, ACK=1	2001	3002	Si 发送 FIN 请求关闭
第 4 次	C→Si	ACK=1	3002	2002	C 确认 Si 的 FIN

序号推算逻辑：

- C 发送数据：起始 seq=1001，长度 2000B → 结束 seq=3001
- C 的 FIN：seq=3001，消耗 1 序号 → Si 确认 ack_seq=3002
- Si 全程未发送数据：seq 始终为 2001
- Si 的 FIN：seq=2001，消耗 1 序号 → C 确认 ack_seq=2002

C 收到的最后一个 TCP 报文段（第 3 次挥手）：

FIN=1, seq=2001, ack_seq=3002

(4) 文件传输总时延计算

时间轴分析：

时间段	过程	耗时	累计时间	关键事件
0~5ms	三次握手	1RTT	5ms	连接建立完成
5~15ms	数据传输	2RTT	15ms	2000B 分 4 段传输完成
15~20ms	四次挥手	1RTT	20ms	C 确认 Si 收到全部数据

数据传输详细过程：

```
第 1 个 RTT (5~10ms):
  - C 发送 2 段(1000B) → Si 确认 → cwnd 增至 3MSS 但受限于 rwnd=1000B
第 2 个 RTT (10~15ms):
  - C 再发送 2 段(1000B) → Si 确认 → 2000B 全部传输完成
```

挥手时延分析：

- 第 1-2 次挥手（C→FIN, Si→ACK）：0.5RTT
- 第 3-4 次挥手（Si→FIN, C→ACK）：0.5RTT
- 总计 1RTT，但 C 确认 Si 成功接收是在收到第 3 次挥手时（Si 的 FIN-ACK）

• C 确定 Si 成功接收文件的时间：4RTT = 20ms

解析：

- 握手 1RTT（连接建立）
- 数据传输 2RTT（每轮发送 1000B 需 1RTT 确认）
- 挥手前半程 0.5RTT（C 发 FIN→Si 确认）
- 挥手后半程 0.5RTT（Si 发 FIN 确认已收完数据）
- 总计：1+2+0.5+0.5=4RTT